In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

In [3]:
```
df = pd.read_csv("AusApparalSales4thQrt2020.csv")
df.head()
```

Out[3]:

	Date	Time	State	Group	Unit	Sales
0	1-Oct-2020	Morning	WA	Kids	8	20000
1	1-Oct-2020	Morning	WA	Men	8	20000
2	1-Oct-2020	Morning	WA	Women	4	10000
3	1-Oct-2020	Morning	WA	Seniors	15	37500
4	1-Oct-2020	Afternoon	WA	Kids	3	7500

In [5]:
```
df.shape
```

Out[5]:
```
(7560, 6)
```

1. Data Wrangling

In [6]:
```
# q1. Ensure that the data is clean and free from any missing or incorrect entri
df.isnull().sum()
```

Out[6]:
```
Date     0
Time     0
State    0
Group    0
Unit     0
Sales    0
dtype: int64
```

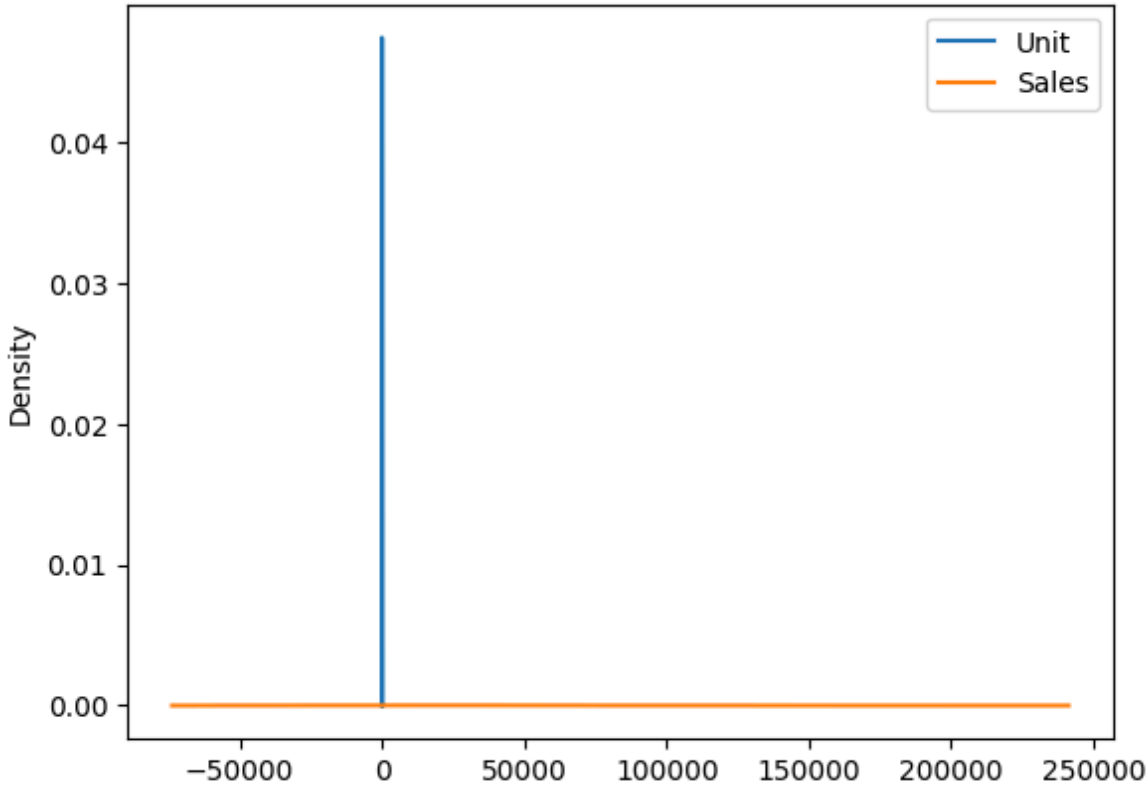
In [7]:
```
# q3. Choose a suitable data wrangling technique—either data standardization or
df.describe()
```

Out[7]:

	Unit	Sales
count	7560.000000	7560.000000
mean	18.005423	45013.558201
std	12.901403	32253.506944
min	2.000000	5000.000000
25%	8.000000	20000.000000
50%	14.000000	35000.000000
75%	26.000000	65000.000000
max	65.000000	162500.000000

In [9]:
```
df[["Unit", "Sales"]].plot.kde()
```

Out[9]:
```
<Axes: ylabel='Density'>
```



In [10]:
```
from sklearn.preprocessing import StandardScaler
```

In [11]:
```
ss = StandardScaler()
```

In [12]:
```
newdf=df[['Unit','Sales']]
```

In [13]:
```
data_transformed = ss.fit_transform(newdf)
```

In [14]:
```
type(data_transformed)
```

Out[14]:
```
numpy.ndarray
```

In [15]:
```
newdf = pd.DataFrame(data_transformed,columns = ['Unit','Sales'])
```

In [16]:
```
newdf
```

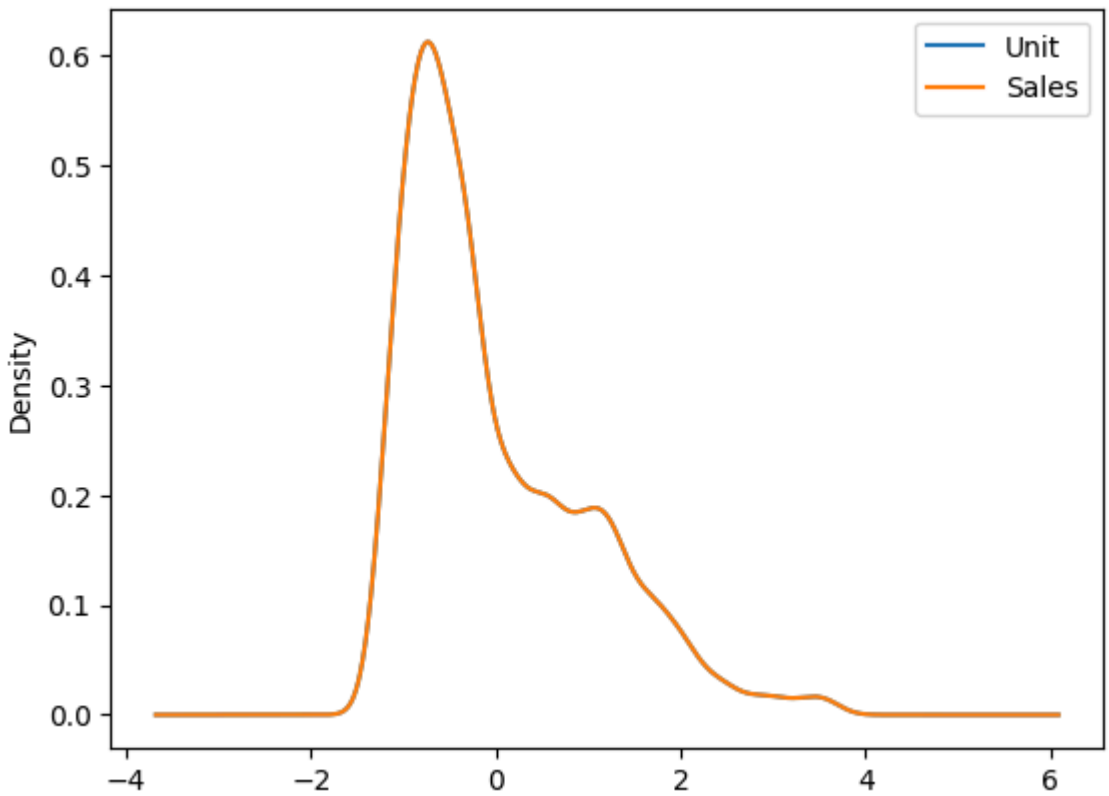
Out[16]:

	Unit	Sales
0	-0.775581	-0.775581
1	-0.775581	-0.775581
2	-1.085645	-1.085645
3	-0.232969	-0.232969
4	-1.163162	-1.163162
...	...	...
7555	-0.310485	-0.310485
7556	-0.232969	-0.232969
7557	-0.232969	-0.232969
7558	-0.543033	-0.543033
7559	-0.388001	-0.388001

7560 rows × 2 columns

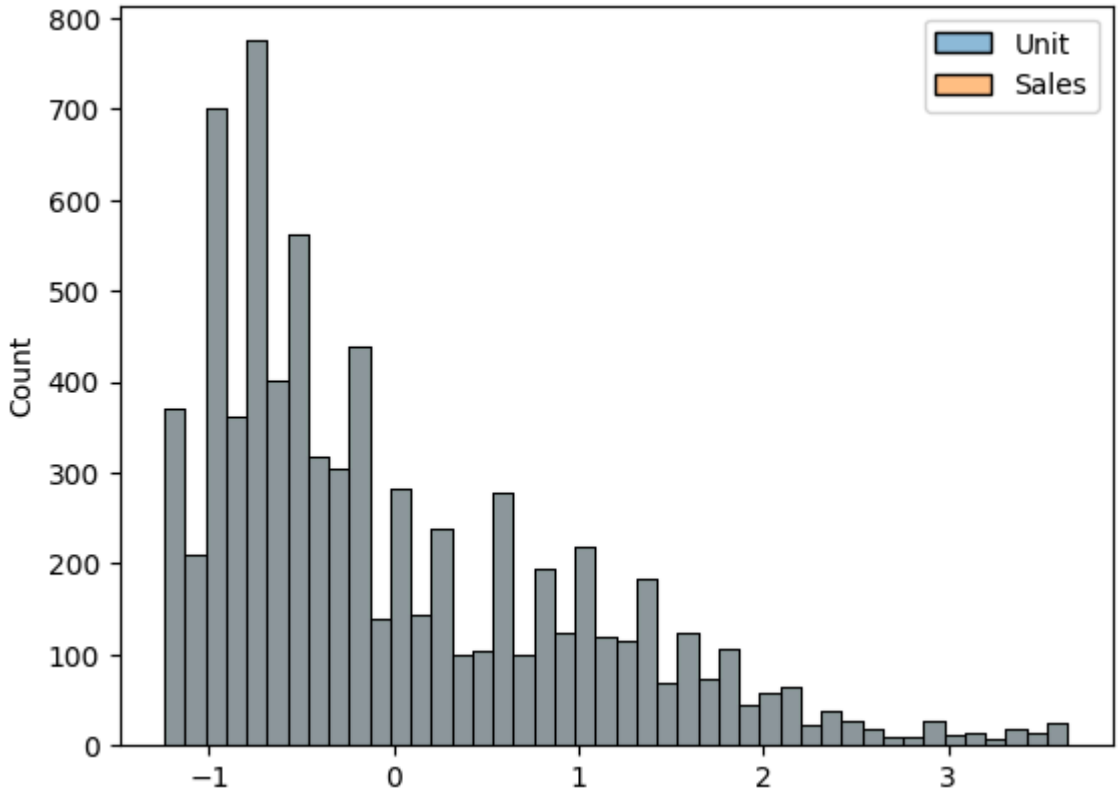
In [17]:
```
newdf.plot.kde()
```

Out[17]:
```
<Axes: ylabel='Density'>
```



In [18]:
```
sns.histplot(newdf)
```

Out[18]:
```
<Axes: ylabel='Count'>
```



In [19]:
```
from sklearn.preprocessing import MinMaxScaler
```

In [20]:
```
mm =MinMaxScaler()
```

In [21]:
```
normdf = mm.fit_transform(newdf)
```

In [22]:
```
testdf = pd.DataFrame(normdf,columns=newdf.columns)
```

In [23]:
```
testdf
```

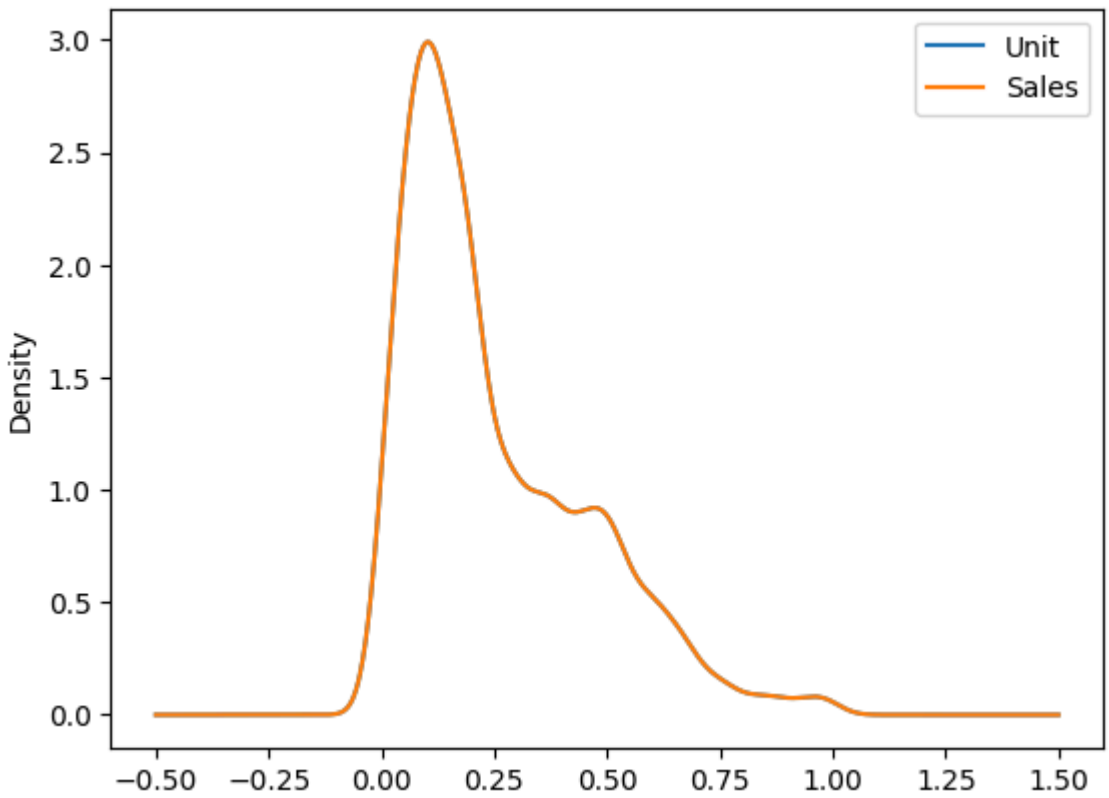
Out[23]:

	Unit	Sales
0	0.095238	0.095238
1	0.095238	0.095238
2	0.031746	0.031746
3	0.206349	0.206349
4	0.015873	0.015873
...	...	...
7555	0.190476	0.190476
7556	0.206349	0.206349
7557	0.206349	0.206349
7558	0.142857	0.142857
7559	0.174603	0.174603

7560 rows × 2 columns

In [24]:
```
testdf.plot.kde()
```

Out[24]:
```
<Axes: ylabel='Density'>
```



In [26]:
```
#ans.standardization and normalization both were presented in the upper section
```

In [27]:
```
# Share your insights regarding the application of the GroupBy() function for ei
df
```

Out[27]:

	Date	Time	State	Group	Unit	Sales
0	1-Oct-2020	Morning	WA	Kids	8	20000
1	1-Oct-2020	Morning	WA	Men	8	20000
2	1-Oct-2020	Morning	WA	Women	4	10000
3	1-Oct-2020	Morning	WA	Seniors	15	37500
4	1-Oct-2020	Afternoon	WA	Kids	3	7500
...	...	...	...	...	...	...
7555	30-Dec-2020	Afternoon	TAS	Seniors	14	35000
7556	30-Dec-2020	Evening	TAS	Kids	15	37500
7557	30-Dec-2020	Evening	TAS	Men	15	37500
7558	30-Dec-2020	Evening	TAS	Women	11	27500
7559	30-Dec-2020	Evening	TAS	Seniors	13	32500

7560 rows × 6 columns

In [28]:
```
state_wise_totalsales=df.groupby('State')["Sales"].sum().sort_values()
```

In [29]:
```
# ans.Merging data of State and Sales to visualize data.
state_wise_totalsales
```

Out[29]:
```
State
WA       22152500
NT       22580000
TAS      22760000
QLD      33417500
SA       58857500
NSW      74970000
VIC     105565000
Name: Sales, dtype: int64
```

2.Data Analysis

In [30]:
```
# Perform descriptive statistical analysis on the data in the Sales and Unit col
newdf = df[["Unit", "Sales"]]
```

In [31]:
```
newdf
```

Out[31]:

	Unit	Sales
0	8	20000
1	8	20000
2	4	10000
3	15	37500
4	3	7500
...	...	...
7555	14	35000
7556	15	37500
7557	15	37500
7558	11	27500
7559	13	32500

7560 rows × 2 columns

In [32]:
```
newdf.mean()
```

Out[32]:
```
Unit        18.005423
Sales    45013.558201
dtype: float64
```

In [33]:
```
newdf.median()
```

Out[33]:
```
Unit        14.0
Sales    35000.0
dtype: float64
```

In [34]:
```
newdf.mode()
```

Out[34]:

	Unit	Sales
0	9	22500

In [35]:
```
newdf.std()
```

Out[35]:
```
Unit        12.901403
Sales    32253.506944
dtype: float64
```

In [36]:
```
# q2. Identify the group with the highest sales and the group with the lowest sa
grouped_df1 = df.groupby('Group')
grouped_df1.apply(lambda x: x.sort_values(by = 'Sales', ascending=False))
```

```
C:\Users\lenovo\AppData\Local\Temp\ipykernel_952\1278928554.py:3: DeprecationWarn
ing: DataFrameGroupBy.apply operated on the grouping columns. This behavior is de
precated, and in a future version of pandas the grouping columns will be excluded
from the operation. Either pass `include_groups=False` to exclude the groupings o
r explicitly select the grouping columns after groupby to silence this warning.
  grouped_df1.apply(lambda x: x.sort_values(by = 'Sales', ascending=False))
```

Out[36]:

Group		Date	Time	State	Group	Unit	Sales
Kids	**7432**	29-Dec-2020	Afternoon	VIC	Kids	65	162500
	6340	16-Dec-2020	Afternoon	VIC	Kids	65	162500
	7180	26-Dec-2020	Afternoon	VIC	Kids	63	157500
	7008	24-Dec-2020	Morning	VIC	Kids	63	157500
	6000	12-Dec-2020	Morning	VIC	Kids	63	157500
...	**...**	...	...	...	...	...	...
Women	**3454**	12-Nov-2020	Evening	WA	Women	2	5000
	3458	12-Nov-2020	Morning	NT	Women	2	5000
	3434	11-Nov-2020	Morning	TAS	Women	2	5000
	3686	14-Nov-2020	Morning	TAS	Women	2	5000
	4878	29-Nov-2020	Afternoon	WA	Women	2	5000

7560 rows × 6 columns

In [37]:
```
grouped_df1.apply(lambda x: x.sort_values(by = 'Sales', ascending=False)).max()
```

```
C:\Users\lenovo\AppData\Local\Temp\ipykernel_952\3175097016.py:1: DeprecationWarn
ing: DataFrameGroupBy.apply operated on the grouping columns. This behavior is de
precated, and in a future version of pandas the grouping columns will be excluded
from the operation. Either pass `include_groups=False` to exclude the groupings o
r explicitly select the grouping columns after groupby to silence this warning.
  grouped_df1.apply(lambda x: x.sort_values(by = 'Sales', ascending=False)).max()
```

Out[37]:
```
Date     9-Oct-2020
Time        Morning
State            WA
Group         Women
Unit             65
Sales        162500
dtype: object
```

In [38]:
```
grouped_df1.apply(lambda x: x.sort_values(by = 'Sales', ascending=False)).min()
```

```
C:\Users\lenovo\AppData\Local\Temp\ipykernel_952\3611652529.py:1: DeprecationWarn
ing: DataFrameGroupBy.apply operated on the grouping columns. This behavior is de
precated, and in a future version of pandas the grouping columns will be excluded
from the operation. Either pass `include_groups=False` to exclude the groupings o
r explicitly select the grouping columns after groupby to silence this warning.
  grouped_df1.apply(lambda x: x.sort_values(by = 'Sales', ascending=False)).min()
```

Out[38]:
```
Date     1-Dec-2020
Time      Afternoon
State           NSW
Group          Kids
Unit              2
Sales          5000
dtype: object
```

In [39]:
```
#ans 2.2 Determine 'women' group is generating the highest sales, and 'kids'␣
#↪group is generating the lowest sales.
from datetime import datetime
from matplotlib import dates
%matplotlib inline
```

In [40]:
```
df['Date'] = pd.to_datetime(df['Date'])
```

In [41]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7560 entries, 0 to 7559
Data columns (total 6 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Date    7560 non-null   datetime64[ns]
 1   Time    7560 non-null   object
 2   State   7560 non-null   object
 3   Group   7560 non-null   object
 4   Unit    7560 non-null   int64
 5   Sales   7560 non-null   int64
dtypes: datetime64[ns](1), int64(2), object(3)
memory usage: 354.5+ KB
```

In [42]:
```
#spliting Date into Day,Month,Year
df['Day'] = pd.DatetimeIndex(df['Date']).day
df['Month']=pd.DatetimeIndex(df['Date']).month
df['Year']=pd.DatetimeIndex(df['Date']).year
```

In [43]:
```
df
```

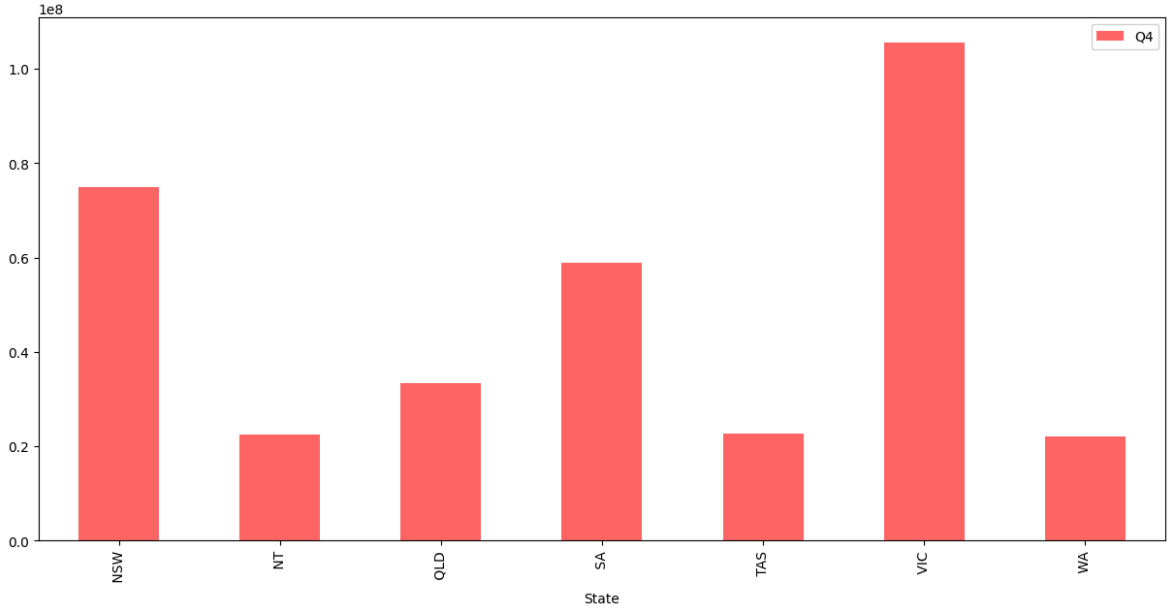
Out[43]:

	Date	Time	State	Group	Unit	Sales	Day	Month	Year
0	2020-10-01	Morning	WA	Kids	8	20000	1	10	2020
1	2020-10-01	Morning	WA	Men	8	20000	1	10	2020
2	2020-10-01	Morning	WA	Women	4	10000	1	10	2020
3	2020-10-01	Morning	WA	Seniors	15	37500	1	10	2020
4	2020-10-01	Afternoon	WA	Kids	3	7500	1	10	2020
...	...	...	...	...	...	...	...	...	...
7555	2020-12-30	Afternoon	TAS	Seniors	14	35000	30	12	2020
7556	2020-12-30	Evening	TAS	Kids	15	37500	30	12	2020
7557	2020-12-30	Evening	TAS	Men	15	37500	30	12	2020
7558	2020-12-30	Evening	TAS	Women	11	27500	30	12	2020
7559	2020-12-30	Evening	TAS	Seniors	13	32500	30	12	2020

7560 rows × 9 columns

In [44]:
```
Q4 = df[(df['Date']>='2020-10-01') & (df['Date']<='2020-12-31')].groupby('State'
```

In [45]:
```
plt.figure(figsize = (15,7))
Q4.plot(kind='bar', alpha = 0.6, color='r', legend = True)
plt.legend(["Q4"])
plt.show()
```



In [46]:
```
df.Date = pd.to_datetime(df.Date)
df.set_index('Date', inplace=True)
```

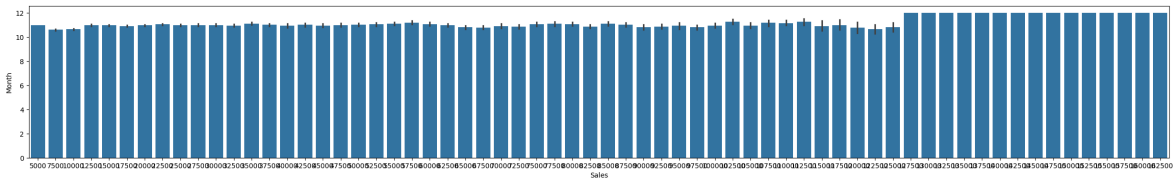
In [47]:
```
df.columns
```

Out[47]:
```
Index(['Time', 'State', 'Group', 'Unit', 'Sales', 'Day', 'Month', 'Year'], dtyp
e='object')
```

In [48]:
```
df_monthly_report = df[['Sales','Month']]
```

In [49]:
```
plt.figure(figsize = (30,4))
sns.barplot(x='Sales', y= "Month", data = df_monthly_report)
```

Out[49]:
```
<Axes: xlabel='Sales', ylabel='Month'>
```

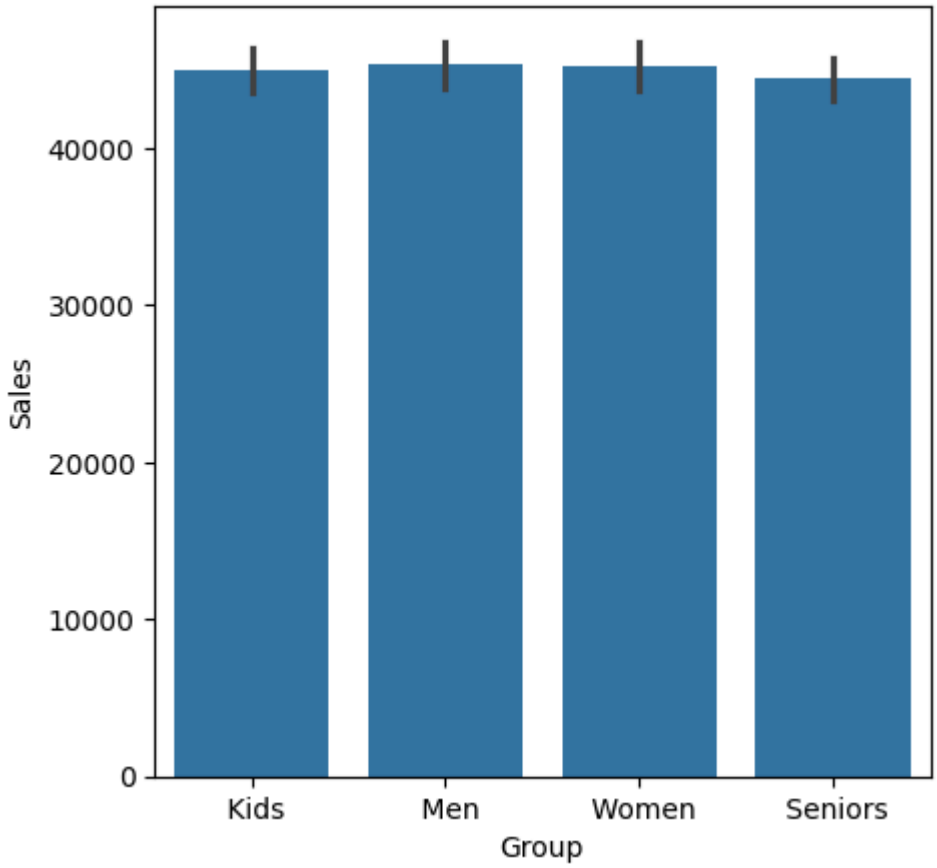


In [50]:
```
#ans. we can clearly see that in the month of december the sales is high.
```

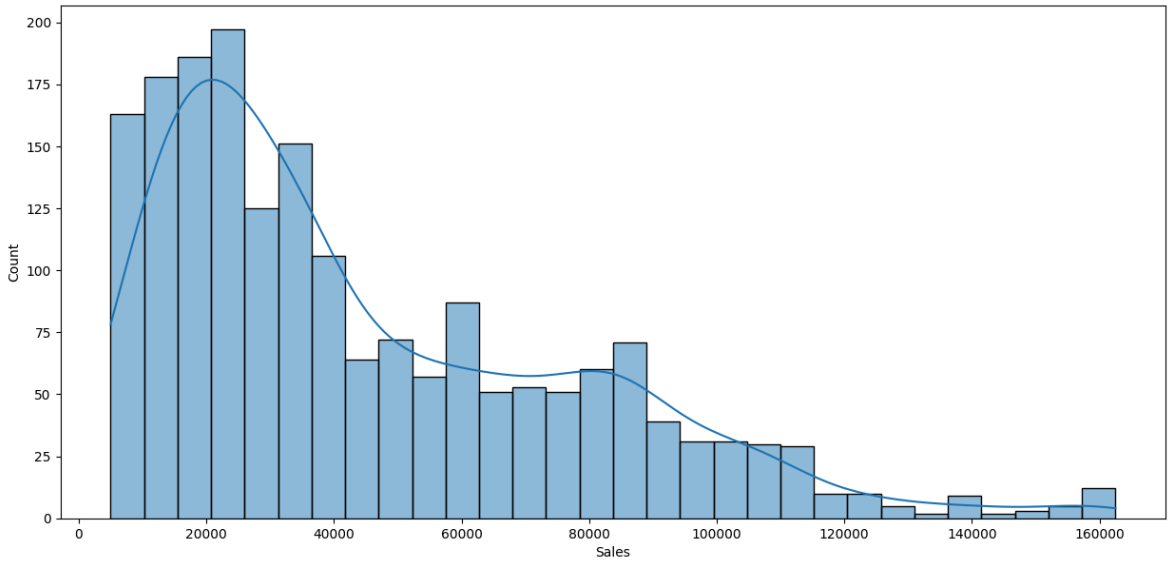
3. Data Visualisation

In [51]:
```
#3.1 State-wise sales analysis for different groups (kids, women, men, and senio
plt.figure(figsize = (5,5))
sns.barplot(x="Group",y="Sales",data = df)
```

Out[51]:
```
<Axes: xlabel='Group', ylabel='Sales'>
```



In [52]:
```
std_data = pd.DataFrame(df.groupby('Group')["Sales"].std().sort_values(ascending
plt.figure(figsize = (15,7))
sns.histplot(df[df['Group'] == std_data.head(1).index[0]]['Sales'], kde =True,bi
plt.show()
```



In [53]:
```
#3.3 Time-of-the-day analysis: during which time of the day are sales the highes
grouped_df1.apply(lambda x: x.sort_values(by = 'Sales', ascending=False)).max()
```

```
C:\Users\lenovo\AppData\Local\Temp\ipykernel_952\3923916024.py:2: DeprecationWarn
ing: DataFrameGroupBy.apply operated on the grouping columns. This behavior is de
precated, and in a future version of pandas the grouping columns will be excluded
from the operation. Either pass `include_groups=False` to exclude the groupings o
r explicitly select the grouping columns after groupby to silence this warning.
  grouped_df1.apply(lambda x: x.sort_values(by = 'Sales', ascending=False)).max()
```

Out[53]:
```
Time      Morning
State          WA
Group       Women
Unit           65
Sales      162500
Day            30
Month          12
Year         2020
dtype: object
```

In [ ]: